

Proiectarea cu Microprocesoare
Power Management pe ATMega324a

Andrei Voinescu

POLITEHNICA University of Bucharest

2020



Outline

Power Management



- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără
  - ADC-ul poate să fie activ



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără
  - ADC-ul poate să fie activ
  - Seriala poate transmite/recepționa



Consumul de energie

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără
  - ADC-ul poate să fie activ
  - Seriala poate transmite/recepționa
  - Circuitele de input GPIO sunt active

- În tehnologia CMOS: $P \sim V_{DD}^2 \cdot f$
  - Tranzițiile costă
- Ce tranziții avem când în cod avem `while(1) ;`?
  - Încă se execută instrucțiuni, PC se modifică etc.
  - Toate timerele active numără
  - ADC-ul poate să fie activ
  - Seriala poate transmite/recepționa
  - Circuitele de input GPIO sunt active
- Power management - procedeul prin care restricționăm ce poate face procesorul

- Mai multe stări de power management
  - Mai puține periferice
  - Mai puține ceasuri active
  - Mai puține metode de revenire
- Periferice dezactivabile manual
  - `PRR0` - Power Reduction Register
    - Un bit pt fiecare periferic

Sleep Mode	Active Clock Domains					Oscillators		Wake-up Sources						
	clk_{CPU}	clk_{FLASH}	clk_{IO}	clk_{ADC}	clk_{ASY}	Main Clock Source Enabled	Timer Oscillator Enabled	INT and PCINT	TWI Address Match	Timer2	SPM/EEPROM Ready	ADC	WDT	Other I/O
Idle			Yes	Yes	Yes	Yes	Yes(2)	Yes	Yes	Yes	Yes	Yes	Yes	Yes
ADC Noise Reduction				Yes	Yes	Yes	Yes(2)	Yes(3)	Yes	Yes(2)	Yes	Yes	Yes	
Power-down								Yes(3)	Yes				Yes	
Power-save					Yes	Yes	Yes(2)	Yes(3)	Yes	Yes			Yes	
Standby(1)						Yes		Yes(3)	Yes				Yes	
Extended Standby					Yes(2)	Yes	Yes(2)	Yes(3)	Yes	Yes			Yes	

Note:

1. Only recommended with external crystal or resonator selected as clock source.
2. If Timer/Counter2 is running in asynchronous mode.
3. For INT2, INT1 and INT0, only level interrupt.

[0]Sursă: Datasheet ATMega324a

Symbol	Parameter	Condition	Min.	Typ.(2)	Max.	Units
I_{CC}	Power Supply Current(1)	Active 1MHz, V_{CC} = 2V	-	0.3	0.55	mA
		Active 4MHz, V_{CC} = 3V	-	1.5	3.5	
		Active 8MHz, V_{CC} = 5V	-	5.2	12	
		Idle 1MHz, V_{CC} = 2V	-	0.06	0.5	
		Idle 4MHz, V_{CC} = 3V	-	0.35	1.5	
		Idle 8MHz, V_{CC} = 5V	-	1.3	5.5	
	Power-save mode(3)	32 kHz TOSC enabled, V_{CC} = 1.8V	-	0.5	-	
		32 kHz TOSC enabled, V_{CC} = 3V	-	0.6	-	
	Power-down mode(3)	WDT enabled, V_{CC} = 3V	-	4.2	15	µA
		WDT disabled, V_{CC} = 3V	-	0.15	3.0	

Note:

1. All bits set in the "PRR – Power Reduction Register ".
2. Typical values at 25°C. Maximum values are test limits in production.
3. The current consumption values include input leakage current.

[0]Sursă: Datasheet ATMega324a

- SMCR - registru control sleep
  - biți SM - modul în care să intre
  - SE - sleep enable (se poate intra in sleep)
- `sleep` - instrucțiunea care intră în sleep efectiv

- Fără ajutor

```
void main ()
{
  while(1)
  {
    do_stuff();
    // ...

    SMCR = (1 << SE); // pentru bitii SM2..0 -- 0 este IDLE
    __asm__ __volatile__ ( "sleep" "\n\t" :: );
    SMCR = 0;
  }
}
```

- Fără ajutor

```
void main ()
{
  while(1)
  {
    do_stuff();
    // ...

    SMCR = (1 << SE); // pentru bitii SM2..0 -- 0 este IDLE
    __asm__ __volatile__ ( "sleep" "\n\t" :: );
    SMCR = 0;
  }
}
```

- Cu ajutor

```
void main ()
{
  while(1)
  {
    do_stuff();
    // ...

    set_sleep_mode(SLEEP_MODE_IDLE);
    sleep_mode();
  }
}
```



Exemple aplicații - Termometru digital

- Un termometru cu afișaj digital
- O dată la 1s măsoară și schimbă afișajul
- Afișaj conectat prin SPI foarte eficient
- Cum minimizăm consumul microcontroller-ului?

- Presupunem un consum mediu de $10\mu A$ pentru senzor și display
- Avem la dispoziție o baterie de ceas CR2032 de $200mAh$ și 3.3V
- Presupunem că putem folosi toată capacitatea
- Avem un cristal de cuarț de 8MHz
- ADC configurat inițial cu prescaler maxim 128

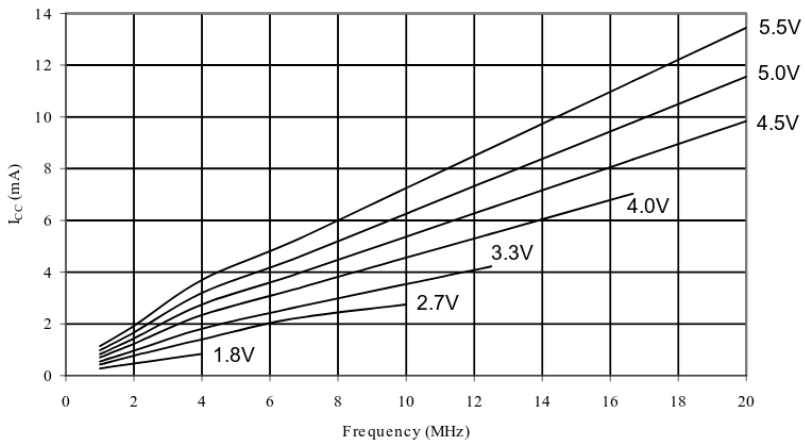
$$t_{conversie} = 13 cicli_{ADC} \cdot 128 \cdot t_{ciclu CPU}$$
$$t_{conversie} = 208\mu s \simeq 200\mu s$$

- Consumul pe SPI presupune schimbul a 100 bytes la 4MHz.

$$t_{comunicare} = \frac{100 * 8}{4000000} s$$
$$t_{comunicare} = 200\mu s$$

- Neglijăm restul operațiilor

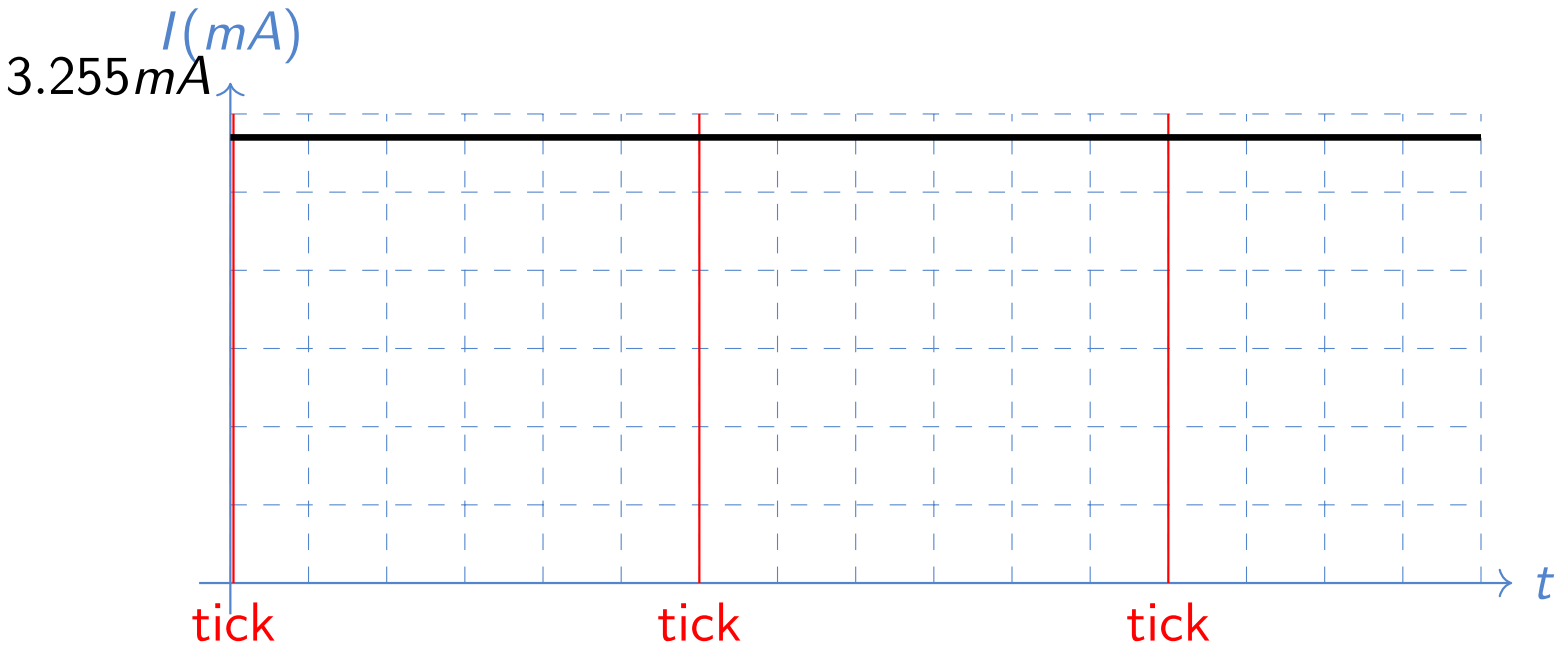
Calcul consum activ ATMega324a



PRR bit	Additional Current consumption compared to Active with external clock (See Figure 30-1 and Figure 30-2)
PRTIM2	2.5%
PRTIM1	1.6%
PRTIM0	0.7%
PRADC	4.2%
PRSPI	2.7%

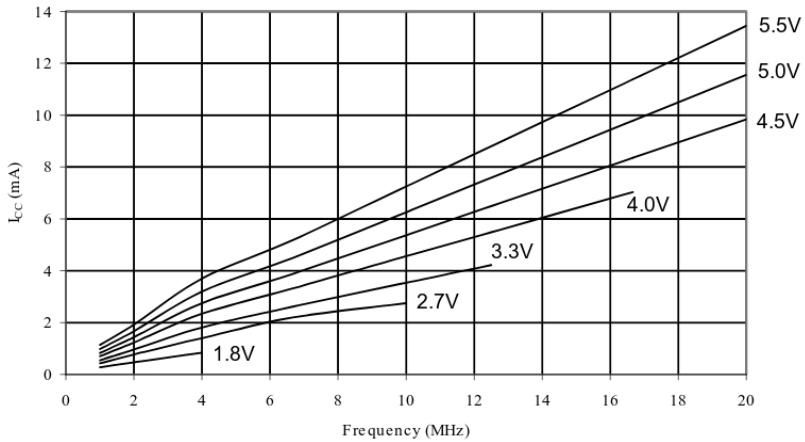
$$I_{CC_{total}} = I_{CC_{Active,8MHz,3.3V}}(1 + P_{ADC} + P_{SPI} + P_{TIM1})$$
$$I_{CC_{total}} = 3mA \cdot (1 + 0.042 + 0.027 + 0.016)$$
$$I_{CC_{total}} = 3mA \cdot (1 + 0.042 + 0.027 + 0.016) = 3.255mA$$

[0]Sursă: Datasheet ATMega324a



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{3.26 + 0.01} \simeq 61.1h$$

Calcul consum activ ATMega324a



PRR bit	Additional Current consumption compared to Active with external clock (See Figure 30-1 and Figure 30-2)
PRTIM2	2.5%
PRTIM1	1.6%
PRTIM0	0.7%
PRADC	4.2%
PRSPI	2.7%

- Dacă folosim PRR0 să închidem ADC și SPI când nu avem nevoie?

$$I_{CC_{conversie}} = I_{CC_{Active,8MHz,3.3V}}(1 + P_{ADC} + P_{TIM1}) = 3.174mA$$
$$I_{CC_{comunicatie}} = I_{CC_{Active,8MHz,3.3V}}(1 + P_{SPI} + P_{TIM1}) = 3.129mA$$
$$I_{CC_{rest}} = I_{CC_{Active,8MHz,3.3V}}(1 + P_{TIM1}) = 3.048mA$$

[0]Sursă: Datasheet ATMega324a

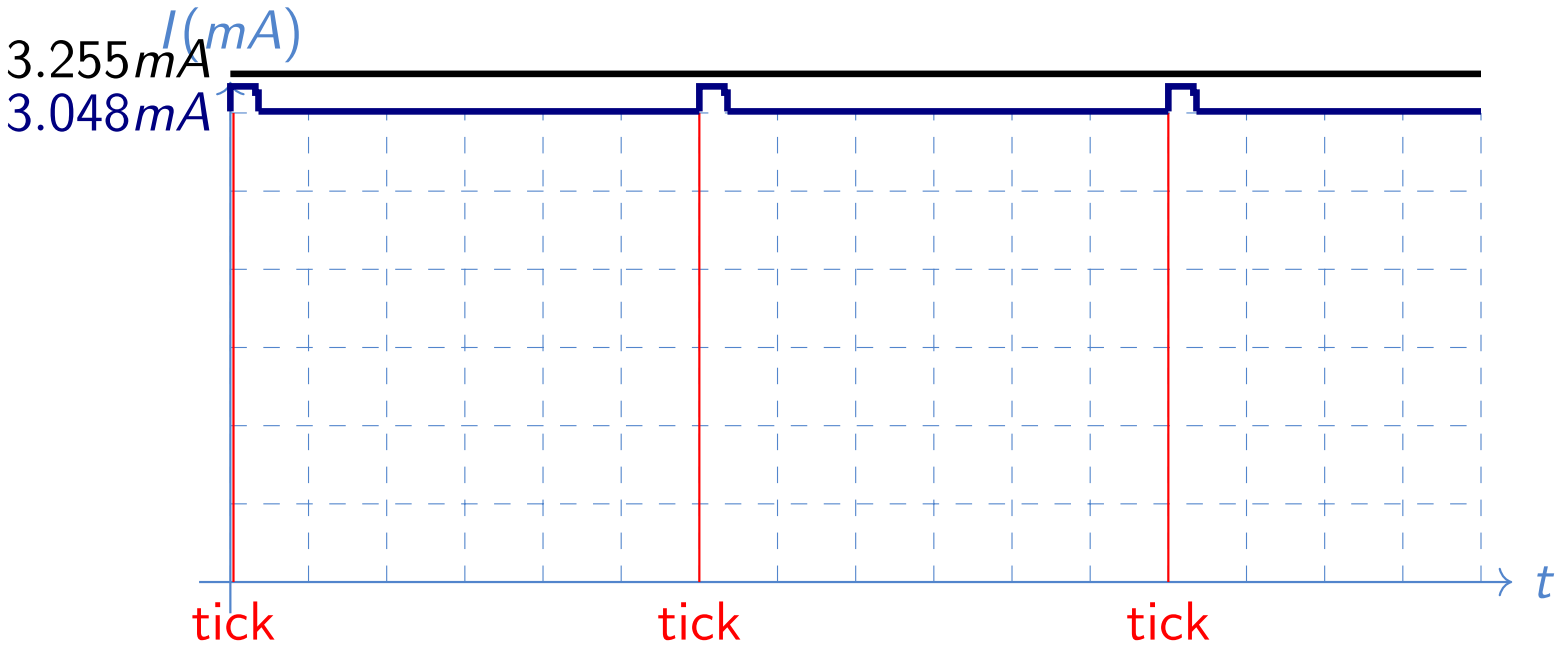
- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$

$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$

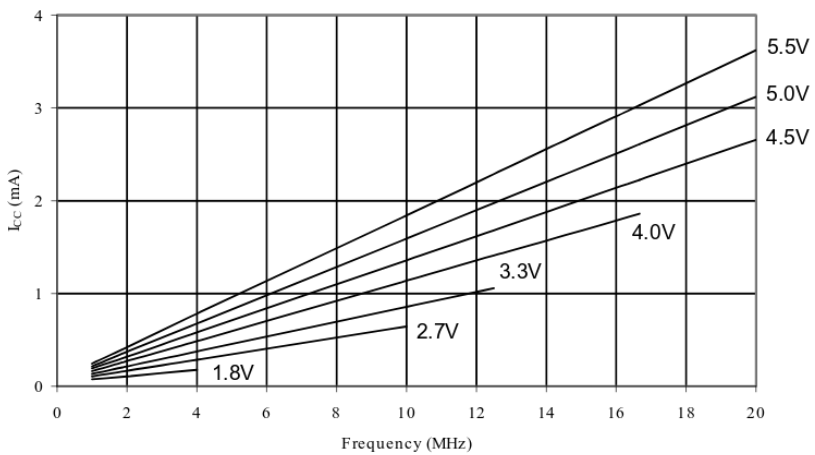
$$I_{CC_{mediu}} = \frac{3.174 \cdot 200 + 3.129 \cdot 200 + 3.048 \cdot 999600}{1000000}$$

$$I_{CC_{mediu}} = 3.048 \mu A$$



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{3.05 + 0.01} \simeq 65.3h$$

- Putem folosi Idle Mode, cu wake-up pe întreruperea de Timer1
- Putem număra maxim 4s (vezi curs Timere) cu Timer1
- Alegem întrerupere de Timer1 la 1s
- Neglijăm timpii de intrare/ieșire din Idle



$$I_{CC_{Idle}} = I_{CC_{Idle,8Mhz,3.3V}} \cdot (1 + P_{TIM1,Idle}) = 0.8 \cdot 1.06 = 0.848mA$$

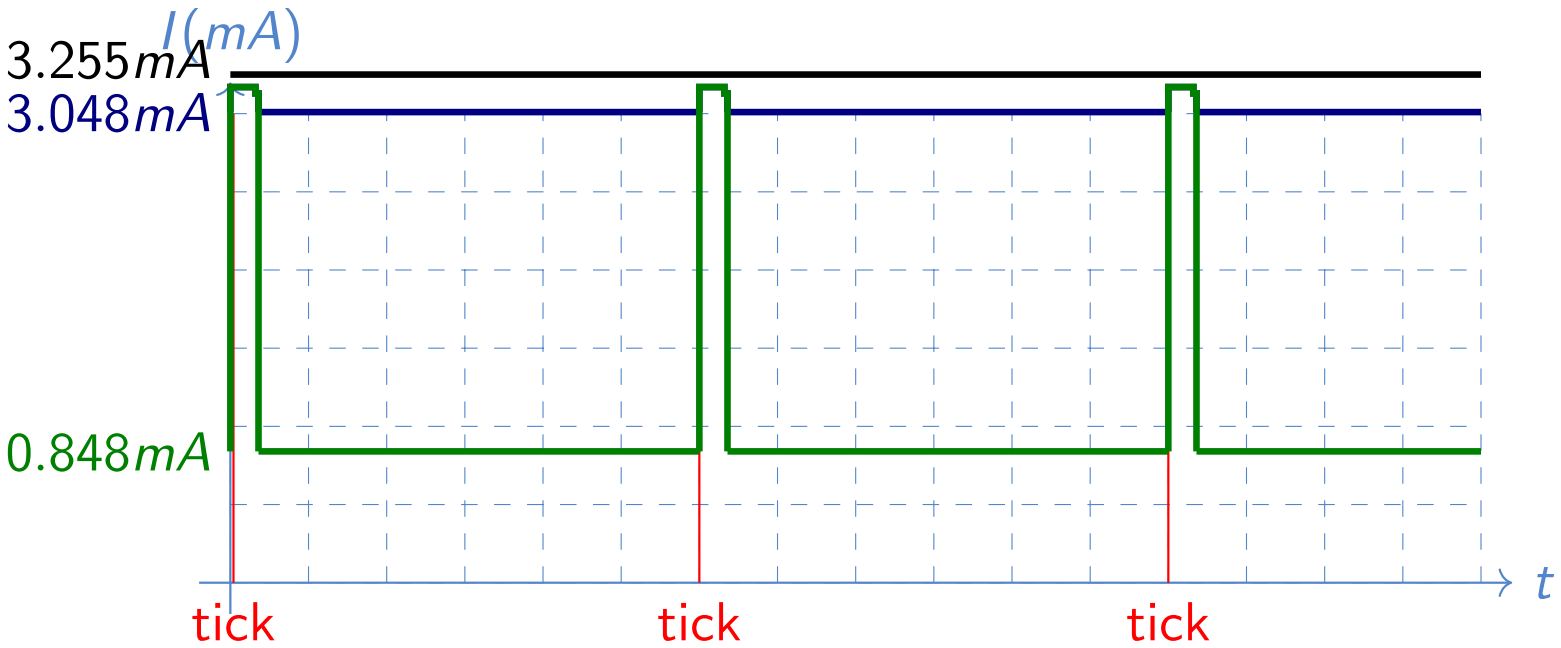
- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$

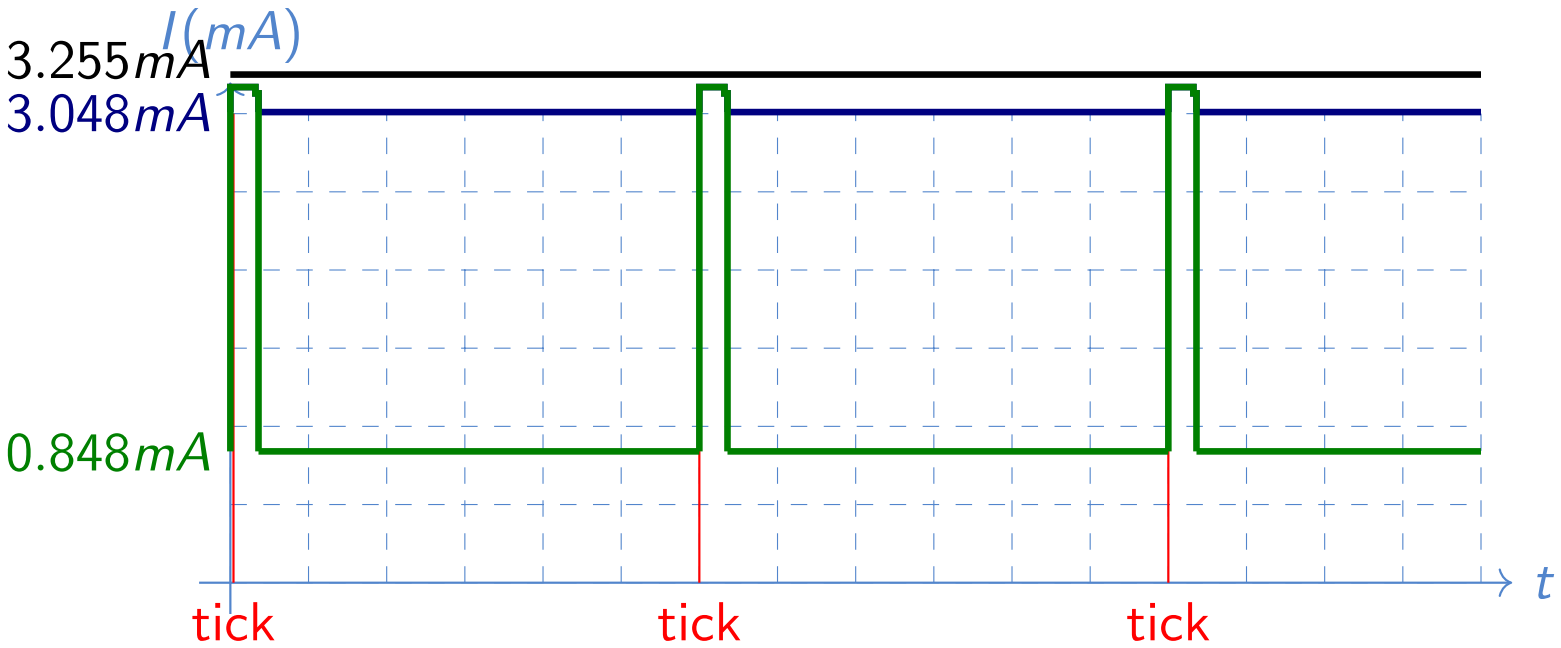
$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$

$$I_{CC_{mediu}} = \frac{3.174 \cdot 200 + 3.129 \cdot 200 + 0.848 \cdot 999800}{1000000}$$

$$I_{CC_{mediu}} = 0.848mA$$



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.85 + 0.01} \simeq 232.56h$$



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.85 + 0.01} \simeq 232.56h$$

- Deja de 4 ori mai bine!

- Power-down - cel mai eficient mod de (ne)lucru
  - Poate fi trezit cu: întrerupere externă, i2c, watchdog
  - Consum la $25^{\circ}C$ de $0.2\mu A$ fără watchdog

- Power-down - cel mai eficient mod de (ne)lucru
  - Poate fi trezit cu: întrerupere externă, i2c, watchdog
  - Consum la $25^{\circ}C$ de $0.2\mu A$ fără watchdog
  - Noi am avea nevoie de watchdog să-l trezim $\rightarrow 4\mu A$
- Power-save - Adăugă timer-ul 2 dacă funcționează cu cristal propriu
  - Consum la $25^{\circ}C$, 3.3V de $0.75\mu A$ fără watchdog

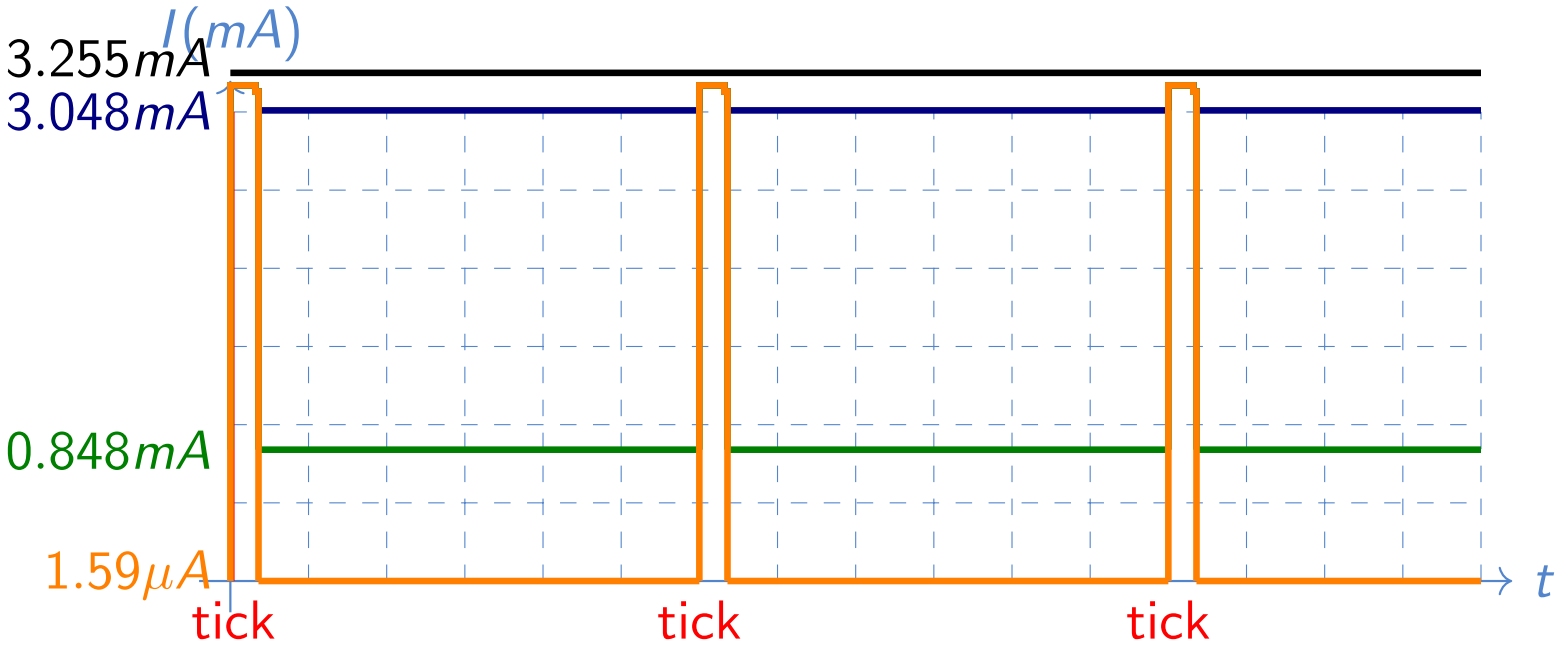
- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$

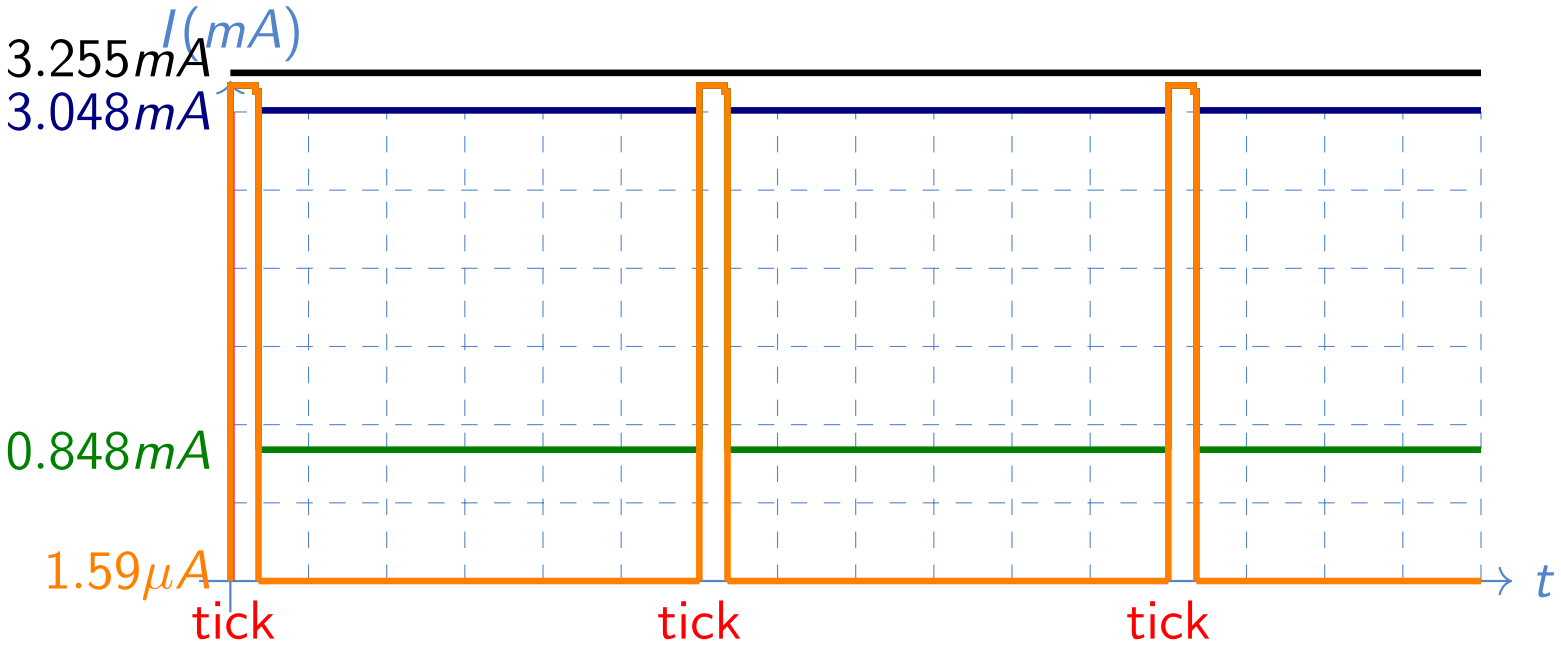
$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$

$$I_{CC_{mediu}} = \frac{3.174 \cdot 200 + 3.129 \cdot 200 + 0.00075 \cdot 999600}{1000000}$$

$$I_{CC_{mediu}} = 1.59 \mu A$$

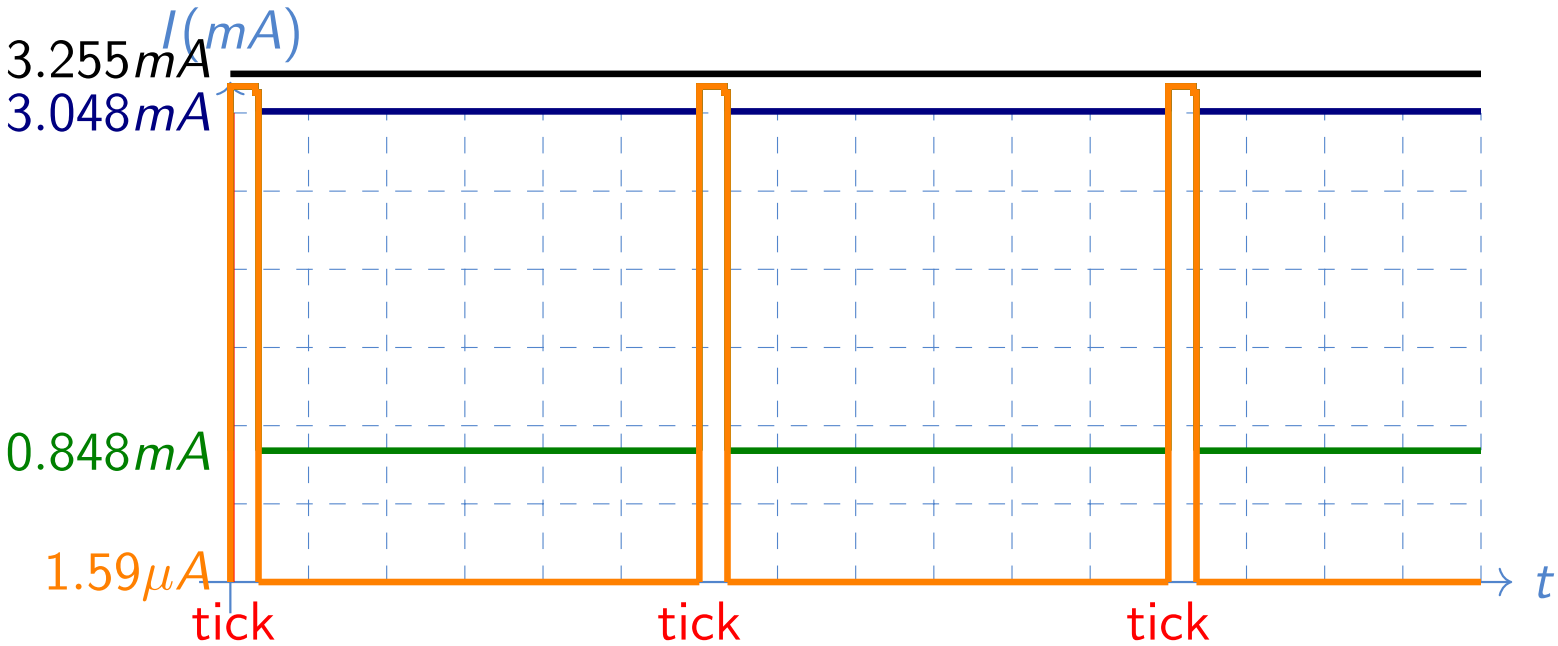


$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.00159 + 0.01} \simeq 17256h \simeq 719zile$$



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.00159 + 0.01} \simeq 17256h \simeq 719zile$$

- De 313 ori mai bine decât estimarea inițială!



$$t_{baterie} = \frac{Capacitate}{Consum} = \frac{200}{0.00159 + 0.01} \simeq 17256h \simeq 719zile$$

- De 313 ori mai bine decât estimarea inițială!
- Consumul MCU-ului este de 2047 ori mai mic

- Senzorul oferă o scară de $10mV/1°C$, cu o precizie de $0.5°C$
- Rezoluția ADC-ului este de 10 biți, la $3.3V \rightarrow 3.22mV$
  - Nu putem reduce rezoluția ADC-ului
  - Frecvența ADC-ului trebuie să fie între 50 și 200kHz
  - Cu PS 128 la 8Mhz este la 62.5kHz $\rightarrow$ putem reduce prescaler-ul de 2 ori
  - Pentru rezoluție mai mică, puteam reduce la PS 16 sau 32

- Am redus prescaler-ul de la 128 la 64

$$t_{conversie} = 13 cicli_{ADC} \cdot 64 \cdot t_{ciclu CPU}$$
$$t_{conversie} = 104\mu s \simeq 100\mu s$$

- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$
$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$
$$I_{CC_{mediu}} = \frac{3.174 \cdot 100 + 3.129 \cdot 200 + 0.00075 \cdot 999700}{10000000}$$
$$I_{CC_{mediu}} = 1.27\mu A$$

- Dar dacă am face totul mai încet?



Optimizări frecvență

- Dar dacă am face totul mai încet?
- 1MHz MCU, SPI 500kHz (minim /2 prescaler)

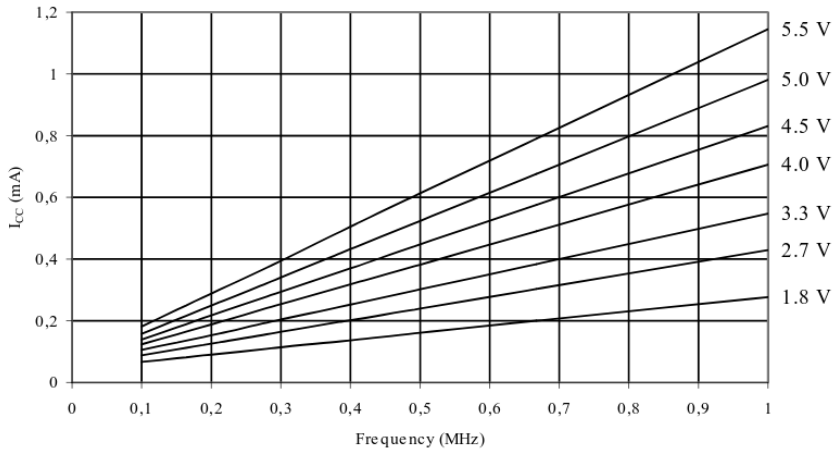
- Dar dacă am face totul mai încet?
- 1MHz MCU, SPI 500kHz (minim /2 prescaler)
- ADC prescaler 8 $\rightarrow$ 125000

$$t_{conversie} = 13 cicli_{ADC} \cdot 8 \cdot t_{ciclu CPU}$$
$$t_{conversie} = 104\mu s \simeq 100\mu s$$

- SPI durează mai mult

$$t_{comunicare} = \frac{100 * 8}{500000} s = 1.6ms$$

Calcul consum activ 1MHz



PRR bit	Additional Current consumption compared to Active with external clock (See Figure 30-1 and Figure 30-2)
PRTIM2	2.5%
PRTIM1	1.6%
PRTIM0	0.7%
PRADC	4.2%
PRSPI	2.7%

$$I_{CC_{total}} = I_{CC_{Active,1MHz,3.3V}}(1 + P_{ADC} + P_{SPI} + P_{TIM1})$$
$$I_{CC_{total}} = 0.55mA \cdot (1 + 0.042 + 0.027 + 0.016) = 0.597mA$$
$$I_{CC_{conversie}} = I_{CC_{Active,1MHz,3.3V}}(1 + P_{ADC} + P_{TIM1}) = 0.588mA$$
$$I_{CC_{comunicatie}} = I_{CC_{Active,1MHz,3.3V}}(1 + P_{SPI} + P_{TIM1}) = 0.573mA$$

[0]Sursă: Datasheet ATMega324a

- Curentul mediu este media ponderată pe timpi

$$I_{CC_{mediu}} = \frac{\sum I_{CC_x} \cdot t_x}{\sum t_x}$$

$$I_{CC_{mediu}} = \frac{I_{CC_{conversie}} \cdot t_{coversie} + I_{CC_{comunicatie}} \cdot t_{comunicatie} + I_{CC_{rest}} \cdot t_{rest}}{t_{total}}$$

$$I_{CC_{mediu}} = \frac{0.588 \cdot 100 + 0.573 \cdot 1600 + 0.00075 \cdot 998300}{1000000}$$

$$I_{CC_{mediu}} = 0.98\mu A$$

- Consumul mediu al MCU-ului de 3300 ori mai mic



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite
- Schimbarea cerințelor pentru optimizări în plus



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite
- Schimbarea cerințelor pentru optimizări în plus
  - Trebuie neapărat la 1s pentru temperatură?



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite
- Schimbarea cerințelor pentru optimizări în plus
  - Trebuie neapărat la 1s pentru temperatură?
  - Trebuie neapărat făcut update la ecran dacă nu s-a schimbat nimic?



Ce mai puteam face?

- Estimarea întregului cod (am ignorat mult)
- Scăderea tensiunii de alimentare (altă baterie?)
- Modelarea consumului pentru nivele de baterie diferite
- Schimbarea cerințelor pentru optimizări în plus
  - Trebuie neapărat la 1s pentru temperatură?
  - Trebuie neapărat făcut update la ecran dacă nu s-a schimbat nimic?
- Optimizări pe părțile ignorate (senzor, LCD)



Concluzii

- Cele mai mari economii - Idle sau Power-save



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite
- Premisa2 - Avem un timp de repaus mare între procesări



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite
- Premisa2 - Avem un timp de repaus mare între procesări
- ATMega324a în general e mai eficient per MIPS la frecvențe mai mari *pentru aceeași tensiune*



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite
- Premisa2 - Avem un timp de repaus mare între procesări
- ATMega324a în general e mai eficient per MIPS la frecvențe mai mari *pentru aceeași tensiune*
  - Depinde și de ce alte operații mai facem



Concluzii

- Cele mai mari economii - Idle sau Power-save
- Premisa - avem deja închise toate perifericele nefolosite
- Premisa2 - Avem un timp de repaus mare între procesări
- ATMega324a în general e mai eficient per MIPS la frecvențe mai mari *pentru aceeași tensiune*
  - Depinde și de ce alte operații mai facem
  - Degeaba reducem frecvența dacă nu reducem și tensiunea